

Video-Skript & Protokoll

Folge 1: Bilderkennung mit lokalem Vision-LLM (qwen2.5vl)

Kanal: Data, Paper & Code

17. September 2026

Inhaltsverzeichnis

1	Struktur & Thematische Gliederung	2
2	YouTube Sprecherskript (Vortragstext)	2
3	Theoretischer Hintergrund	3
3.1	Warum automatische Bilderkennung?	3
3.2	VLM-Architektur: Wie Modelle Bilder verstehen	4
3.3	Warum Base64-Konvertierung?	4
4	System-Setup & Ollama Anleitung	5
5	Vollstaendiges Python-Skript	5
6	Ergebnis & Modellausgabe	8
7	Ausblick & Anwendungsbereiche	8

1 Struktur & Thematische Gliederung

Nr.	Themenblock	Kerninhalt & Zielsetzung
01	Begrüßung	Unkonventioneller Einstieg & Channel-Intro.
02	Automatische Bilderkennung	Warum visuelle Datenanalyse in Datenpipelines essenziell ist.
03	Was ist ein LLM	Funktionsweise und Grenzen klassischer Textmodelle.
04	Multimodale LLMs	Wie Sprachmodelle lernen, Bilder als Input zu verarbeiten.
05	Lokale LLMs	Datenschutz, Unabhängigkeit und Hardware-Anforderungen.
06	Ollama Setup	Installation und Betrieb von Ollama im Hintergrund.
07	Modell-Auswahl	Bereitstellung des Multimodal-Modells <code>qwen2.5v1</code> .
08	Python-Integration	REST-API Aufruf (<code>/api/generate</code>) via <code>requests</code> .
09	Base64-Kodierung	Warum und wie Bilder in ASCII-Text umgewandelt werden.
10	Architecture Deep Dive	VLM Training: Vision Encoder (ViT/CLIP), Projection Layers & Text-LLM.
11	Setup & Model Download	Installation von Ollama & Herunterladen von <code>qwen2.5v1</code> .
12	Praxis & Live-Code	Automatisierte Erstellung und Analyse eines Financial Charts.

Tabelle 1: Ablaufplan der ersten Videofolge

2 YouTube Sprecherskript (Vortragstext)

Teil 1: Der unkonventionelle Hook & Intro

[Kamera: Nahaufnahme | Dynamisch & aufdenkend]

„Stellt euch vor, ihr drückt einer KI ein 500-seitiges Buch in die Hand — und innerhalb von zwei Sekunden fasst sie euch das komplette Werk perfekt zusammen. Aber sobald ihr ihr ein simples Bild von einer Fieberkurve oder einem Aktienchart zeigt, sagt sie plötzlich: *‘Ich bin leider nur ein Textmodell, ich habe keine Augen.’*“

Verrückt, oder? Wir füttern KIs mit Terabytes an Text, aber vor einfachen Bildern standen sie jahrelang wie ein Ochse vorm Scheunentor.

Schluss damit. Willkommen bei **Data, Paper & Code**! Heute brechen wir dieses Paradigma auf. Wir bauen uns eine eigene, komplett lokal laufende Bilderkennung auf eurem Rechner. Kein Cloud-Abo, keine Datenlecks nach U.S.A., keine API-Kosten. Nur Python, Ollama und das brandneue `qwen2.5v1`. Schnallt euch an, wir legen direkt los!“

Teil 2: Theorie — Wie eine KI Bilder „frisst“

[Bildschirm: Einblendung Architektur-Diagramm / ViT Patches]

„Bevor wir den Code anwerfen, müssen wir eine fundamentale Hürde verstehen: Ein LLM versteht nur Tokens, also Zahlenvektoren für Wörter. Ein Bild ist aber ein Haufen Pixel.“

Wie bringen wir ein Textmodell dazu, Pixel zu ‘sehen’?

Wir nutzen drei Bausteine: 1. Den **Vision Encoder** — genauer gesagt ein Vision Transformer (ViT). Der zerschneidet unser Bild in ein Schachbrettmuster aus kleinen Kacheln, sogenannten Patches. Jedes Patch wird wie ein eigenes Wort behandelt. 2. Die **Projection Layer** — das ist das mathematische Bindeglied. Sie uebersetzt die Bild-Vektoren exakt in die Sprache des Sprachmodells. 3. Das **Autoregressive LLM** — das am Ende beide Welten kombiniert und uns Wort für Wort erklärt, was auf dem Bild zu sehen ist.“

Teil 3: Installation & Modell-Setup (Ollama)

[Bildschirm: Browser ollama.com / Terminal]

„Damit das Ganze auf unserem eigenen PC läuft, brauchen wir zwei Dinge: Die Engine und das Modell.

1. ****Ollama installieren:**** Geht auf `ollama.com`, ladet euch den Installer für Windows, Mac oder Linux herunter und führt ihn aus. Sobald Ollama läuft, seht ihr unten rechts in eurer Taskleiste das kleine Lama-Icon. Das bedeutet: Euer lokaler API-Server läuft auf Port `11434`.
2. ****Vision-Modell ziehen:**** Öffnet jetzt ein Terminal oder die PowerShell und tippt exakt folgenden Befehl ein:

```
ollama run qwen2.5v1
```

Ollama lädt das multimodale `qwen2.5v1`-Modell automatisch herunter und richtet es ein. Wenn das erledigt ist, steht die Schnittstelle bereit und wir können direkt in Python einsteigen!“

Teil 4: Der Base64-Trick & Live Code

[Bildschirm: VS Code / IDE geöffnet]

„Jetzt kommt der Praxistrick. Wenn wir eine REST-API anfahren, wollen wir keine komplizierten Dateiuploads verwalten. Wir nehmen die Bilddatei, jagen sie durch eine **Base64-Kodierung** und machen aus dem Binaerbild einen reinen Text-String aus ASCII-Zeichen. Diesen String packen wir direkt in das JSON-Payload unserer Anfrage an Ollama. Schauen wir uns den Python-Code an: Wir laden Marktdaten via `yfinance`, generieren mit `matplotlib` ein Chart, kodieren es in Base64 und schicken es an `http://localhost:11434/api/generate`.

Lassen wir das Skript laufen... und BUMM! Schaut euch die Antwort im Terminal an: Das Modell hat nicht nur den Aufwaertstrend erkannt, sondern nennt uns exakt die Ausbruchszonen bei den Marken 50 und 70. Vollautomatisch, in Sekunden, auf eurem lokalen Prozessor!“

3 Theoretischer Hintergrund

3.1 Warum automatische Bilderkennung?

In wissenschaftlichen Arbeiten, Finanzreports und Messreihen liegen Daten oft in visueller Form vor (Charts, PDFs, Diagramme). Eine automatische Analyse erlaubt die nahtlose Extraktion von Information ohne manuelles Ablesen.

3.2 VLM-Architektur: Wie Modelle Bilder verstehen

Vision LLMs (VLMs) wie `qwen2.5v1` kombinieren visuelle Signalverarbeitung mit natuerlicher Sprachverarbeitung. Dies geschieht in drei Kernschritten:

1. Vision Encoder (Extraktion visueller Merkmale):

- **ViT (Vision Transformer):** Zerschneidet das Eingabebild in ein Raster kleiner Kacheln (Patches, z. B. 16×16 Pixel). Jedes Patch wird wie ein Wort-Token behandelt, sodass Self-Attention-Mechanismen räumliche Beziehungen im Bild erlernen koennen.
- **CLIP (Contrastive Language-Image Pre-training):** Ein Modell, das darauf trainiert wurde, Bild- und Text-Embeddings in demselben Vektorraum auszurichten, um die inhaltliche Semantik eines Bildes in Bezug auf Sprache zu verstehen.

2. Projection Layer / Cross-Attention (Die Schnittstelle):

Da Vision Encoder und Sprachmodell in unterschiedlichen Vektorraeumen arbeiten, dient dieser Teil als mathematisches Verbindungsglied:

- **Projection Layer (MLP):** Agiert als „Uebersetzer“, der die Vektordimensionen des Vision Encoders (z. B. 768) exakt auf die Erwartung des Text-LLMs (z. B. 4096) projiziert, sodass Bild-Tokens wie visuelle Woerter verarbeitet werden.
- **Cross-Attention:** Verknuepft Text- und Bild-Tokens dynamisch. Die Text-Tokens stellen die Abfrage (Query), waehrend die Bild-Tokens Schluessel und Werte (Keys/Values) liefern. Dadurch richtet das Modell seine Aufmerksamkeit gezielt auf die relevanten Bildbereiche (z. B. den Kursverlauf im Chart).

3. Autoregressives LLM (Generierung der Antwort):

Das Sprachmodell verarbeitet die uebersetzten Bild-Tokens zusammen mit dem Text-Prompt im selben Kontextfenster und generiert Wort fuer Wort die finale Analyse.

3.3 Warum Base64-Konvertierung?

HTTP-REST-APIs senden JSON-Payloads, die prinzipiell aus Text bestehen. Da rohe Bild-Binaerdaten Sonderzeichen enthalten, die das JSON-Format beschaedigen wuerden, wandelt man das Bild in einen **Base64-ASCII-String** um.

Die technischen Hintergruende im Detail:

- **JSON-Kompatibilitaet & Syntax-Schutz:** Binaerdateien (z. B. PNG oder JPEG) enthalten beliebige Bytewerte ($0x00$ bis $0xFF$), darunter Null-Bytes, Steuerzeichen sowie Anfuhrungszeichen (") und Backslashes (\). Diese wuerden das JSON-Format korrumpieren oder Parsing-Fehler ausloesen.
- **Sichere 7-Bit-ASCII-Uebertragung:** Base64 wandelt je 3 Bytes Binaerdaten (24 Bit) in 4 druckbare Zeichen um. Der Zeichenvorrat beschraenkt sich auf 64 neutrale Zeichen (A-Z, a-z, 0-9, +, / sowie = als Padding), die ueber HTTP-Netzwerkverbindungen garantiert nicht veraendert werden.
- **Single-Payload-Struktur:** Das Bild kann als einfacher String-Wert direkt im JSON-Body (`{\"images\": [\"iVBORw0KG...\"]}`) uebertragen werden, ohne dass auf komplexere Datentypen oder `multipart/form-data`-Headers zurueckgegriffen werden muss.
- **Trade-off (Overhead):** Durch das 3-zu-4-Byte-Schema waechst die uebertragene Datenmenge um ca. 33 %. Fuer API-Prompts und einzelne Charts ist dieser Overhead jedoch vernachlaessigbar.

```

1 import base64
2
3 def encode_image_to_base64(image_path: str) -> str:
4     """Wandelt ein lokales Bild in einen Base64-String um."""
5     with open(image_path, "rb") as image_file:
6         return base64.b64encode(image_file.read()).decode('utf-8')

```

Listing 1: Stand-Alone Funktion: Bild zu Base64 konvertieren

4 System-Setup & Ollama Anleitung

Anleitung: Ollama Starten & Einrichten

1. **Download & Installation:** Ollama von ollama.com herunterladen und installieren.
2. **Status pruefen:** Ollama-Icon im System Tray (Taskleiste) pruefen.
3. **Manuell starten:** Falls inaktiv, Terminal oeffnen und `ollama serve` ausfuehren.
4. **Modell laden:** In der Console einmalig ausfuehren:

```
ollama run qwen2.5vl
```

5. **API-Schnittstelle:** Das Python-Skript greift auf <http://localhost:11434/api/generate> zu.

5 Vollstaendiges Python-Skript

Das folgende Skript generiert ein Chart (z. B. Finanzdaten), kodiert es als Base64 und sendet es an das lokale qwen2.5vl-Modell.

```

1 import base64
2 import os
3 import matplotlib.pyplot as plt
4 import requests
5 import yfinance as yf
6
7
8 # =====
9 # 1. BILD ERSTELLEN (z.B. DAX-Chart)
10 # =====
11 def generate_chart_image(ticker: str = "VUSD.L", filename: str = "chart.
12     png") -> str:
13     """Laedt Boersendaten, erstellt ein Chart-Bild und speichert es ab."""
14     data = yf.download(ticker, period="3mo")
15
16     plt.figure(figsize=(10, 5), dpi=150)
17     plt.plot(data.index, data['Close'], label=f"{ticker} Kursverlauf",
18             color='#1f77b4', lw=2)
19     plt.title(f"Aktueller Trend fuer {ticker}", fontsize=14, fontweight='
20     bold')
21     plt.xlabel("Datum")

```

```

19     plt.ylabel("Kurs")
20     plt.grid(True, linestyle='--', alpha=0.6)
21     plt.tight_layout()
22
23     plt.savefig(filename)
24     plt.close()
25     print(f"[INFO] Chart als Bild gespeichert: {filename}")
26     return filename
27
28
29 # =====
30 # 2. BILD ZU BASE64 KONVERTIEREN
31 # =====
32 def encode_image_to_base64(image_path: str) -> str:
33     """Wandelt ein lokales Bild in einen Base64-String um."""
34     with open(image_path, "rb") as image_file:
35         return base64.b64encode(image_file.read()).decode('utf-8')
36
37
38 # =====
39 # 3. BILD AN DAS VISION-LLM IN OLLAMA SENDEN
40 # =====
41 def analyze_chart_with_ollama(
42     image_path: str,
43     ollama_url: str = "http://localhost:11434/api/generate",
44     model_name: str = "qwen2.5vl"
45 ) -> str:
46     """Sendet das Bild an das in Ollama geladene Vision-Modell."""
47
48     # Bild in Base64 umwandeln
49     base64_image = encode_image_to_base64(image_path)
50
51     prompt_text = (
52         "Du bist ein technischer Chart-Analyst. Analysiere das angehaengte  

53         Chart-Bild:\n"
54         "1. Welcher Trend (Aufwaerts, Abwaerts, Seitwaerts) ist erkennbar  

55         ?\n"
56         "2. Identifiziere auffaellige Ausbrueche oder Konsolidierungen im  

57         Verlauf.\n"
58         "3. Gib eine praegnante Zusammenfassung der visuellen Lage in 3  

59         Saetzen."
60     )
61
62     payload = {
63         "model": model_name,
64         "prompt": prompt_text,
65         "images": [base64_image],
66         "stream": False,
67         "options": {
68             "temperature": 0.2
69         }
70     }
71
72     try:
73         print(f"\n[INFO] Sende Bild an Ollama ({model_name})...")
74         response = requests.post(ollama_url, json=payload)
75         response.raise_for_status()
76
77         result_json = response.json()

```

```

74         return result_json.get("response", "Keine Antwort vom Modell
erhalten.")
75
76     except requests.exceptions.RequestException as e:
77         return f"[FEHLER]: Anfrage an Ollama fehlgeschlagen: {e}"
78
79
80 def print_ollama_instructions():
81     instructions = """
82     =====
83     ANLEITUNG: OLLAMA STARTEN & EINRICHTEN
84     =====
85     1. OLLAMA IN DER TASKLEISTE PRUEFEN:
86         - Schau unten rechts in der Windows-Taskleiste (neben der Uhr).
87         - Ist das Ollama-Icon (Lama) sichtbar? -> Ollama laeuft bereits!
88
89     2. OLLAMA MANUELL STARTEN (falls nicht aktiv):
90         - Option A: Oeffne das Windows-Startmenue und klicke auf "Ollama".
91         - Option B: Oeffne ein beliebiges Terminal / PowerShell und tippe:
92             ollama serve
93
94     3. MODELL EINMALIG HERUNTERLADEN (falls noch nicht geschehen):
95         - Oeffne eine PowerShell und fuehre folgenden Befehl aus:
96             ollama run qwen2.5vl
97
98     4. SKRIPT-VORAUSSETZUNG:
99         - Das Python-Skript sendet die Anfragen an: http://localhost:11434
100        - Solange Ollama im Hintergrund laeuft, wird die Bildanalyse
101          erfolgreich verarbeitet.
102    =====
103    """
104    print(instructions)
105
106
107    # =====
108    # EXECUTION BEISPIEL
109    # =====
110    if __name__ == "__main__":
111        # Anleitung vorab auf der Konsole ausgeben
112        print_ollama_instructions()
113
114        # 1. Bildpfad definieren oder Chart generieren
115        chart_file = "Kurve.jpg"
116
117        if not os.path.exists(chart_file):
118            chart_file = generate_chart_image(ticker="VUSD.L", filename="chart
.png")
119
120        # 2. Grafik von Ollama auswerten lassen
121        analysis_result = analyze_chart_with_ollama(chart_file)
122
123        print("\n" + "=" * 50)
124        print("  VISUELLE ERKENNUNG DES LLM (OLLAMA):")
125        print("=" * 50)
126        print(analysis_result)

```

Listing 2: Vollstaendiger Quellcode zur automatischen Chart-Analyse

6 Ergebnis & Modellausgabe

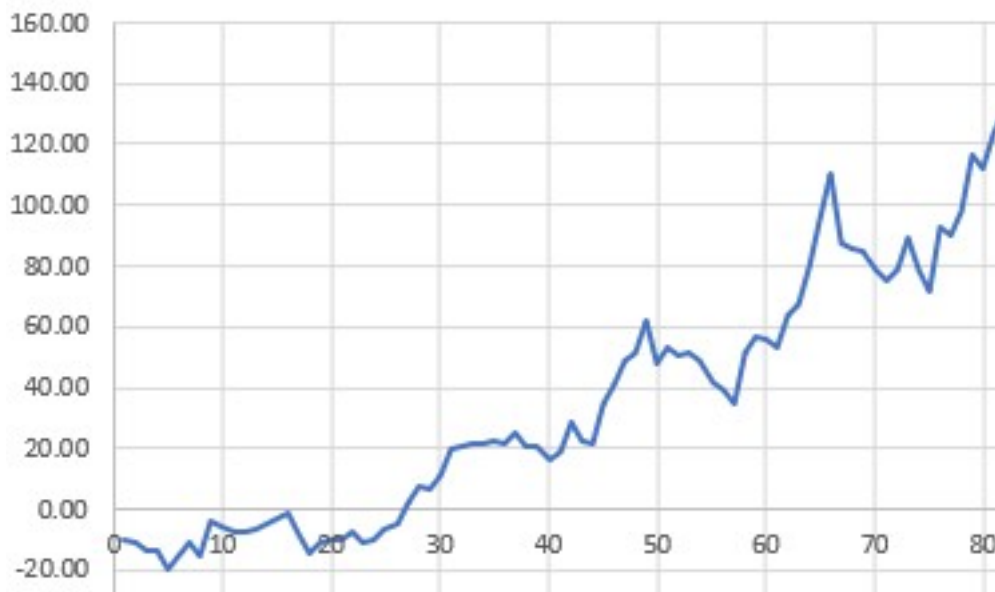


Abbildung 1: Zu analysierender Chart.

Ausgabe im Terminal nach Ausfuehrung des Skripts

Befehl: python Image01.py

[INF0] Sende Bild an Ollama (qwen2.5v1)...

VISUELLE ERKENNUNG DES LLM (OLLAMA):

1. Der Trend des angehaengten Charts ist **Aufwaerts**. Die Kurve steigt kontinuierlich an, was auf eine positive Entwicklung hinweist.
2. **Auffaellige Ausbrueche:** Ein signifikanter Anstieg findet sich zwischen den Marken 50 und 60, gefolgt von einem weiteren starken Aufwaertsschub nach der Marke 70. Diese Punkte koennten als wichtige Unterstuetzungspunkte oder Resistenzlinien dienen.
3. **Zusammenfassung:** Das Chart zeigt eine klare Tendenz zur Steigerung, mit zwei markanten Ausbruechen bei den Marken 50 und 70. Die Kurve ist insgesamt steigend, was auf eine positive Entwicklung hinweist.

7 Ausblick & Anwendungsbereiche

Die erfasste Messkurve und deren mathematische Auswertung bieten eine wesentliche Grundlage fuer praktische Anwendungen in Forschung und Industrie:

- **Modellvalidierung & Simulation:** Abgleich von theoretischen Simulationsmodellen mit realen Messdaten zur Identifikation von Systemgrenzen und Parameterabweichungen.
- **Qualitaetssicherung & Prozessueberwachung:** Festlegung von Toleranzbaendern und Referenzkurven, um Fertigungsfehler oder Materialdefekte fruehzeitig zu erkennen.
- **Regressionsdiagnostik & Fehleranalyse:** Durch die Untersuchung der Residuen (z. B. via

Normalized Residual Plots) lassen sich systematische Messfehler und Sensor-Drifts identifizieren.

- **Integration in KI- & RAG-Pipelines:** Extraktion der Kurvenverläufe und Parameter zur automatisierten Dokumentation, Mustererkennung und Hypothesengenerierung in Analyse-Systemen.